

UNIVERSITÉ LIBRE DE BRUXELLES

Algorithmique 2 – INFO-F-203

Détection de communautés dans DBLP



Jawad Cherkaoui 576517- Rayan Rabeh 576232

Année académique 2025–20265

Table des matières

1	Introduction	2
2	Méthodologie	2
2.1	Tâche 1	2
2.2	Tâche 2	3
3	Résultats	3
3.1	Tâche 1	3
3.2	Tâche 2	5
4	Discussion	5
5	Conclusion	7

1 Introduction

Ce projet analyse le fichier DBLP afin de détecter des communautés d’auteur·rice·s à partir de leurs publications. Comme le fichier est volumineux, le traitement doit se faire publication par publication. Deux notions de communauté sont étudiées.

Dans la tâche 1, on utilise un graphe non orienté de co-publication : deux auteur·rice·s sont relié·e·s s’ils apparaissent ensemble dans une publication.

Dans la tâche 2, on utilise un graphe orienté plus restrictif : on compte les relations du premier auteur vers les autres auteur·rice·s, puis on garde seulement les arêtes qui apparaissent au moins six fois. Les communautés sont alors les composantes fortement connexes de ce graphe filtré.

2 Méthodologie

2.1 Tâche 1

Pour la tâche 1, les communautés correspondent aux composantes connexes d’un graphe non orienté. Nous avons utilisé une structure Union-Find, adaptée au maintien dynamique de composantes. Chaque auteur·rice reçoit un identifiant entier, puis, pour chaque publication, le premier auteur est fusionné avec chacun des autres co-auteur·rice·s. Par transitivité, tous les auteur·rice·s de la publication se retrouvent donc dans la même communauté.

L’Union-Find utilise la compression de chemin et l’union par taille, deux optimisations vues dans le cours pour rendre les opérations presque constantes en pratique. Les publications sont traitées une par une, donc les communautés et leurs tailles sont maintenues en ligne. Les auteur·rice·s répété·e·s dans une même publication sont ignoré·e·s afin d’éviter des unions inutiles.

Si P est le nombre de publications, k le nombre moyen d’auteur·rice·s par publication et n le nombre d’auteur·rice·s distinct·e·s, la complexité est $O(P \cdot k \cdot \alpha(n))$, où α est la fonction d’Ackermann inverse. La mémoire est linéaire en le nombre d’auteur·rice·s distinct·e·s.

2.2 Tâche 2

Pour la tâche 2, nous comptons en ligne les paires orientées $A \rightarrow B$, où A est le premier auteur d’une publication et B un des autres auteur·rice·s. Après le parsing, nous construisons le graphe orienté en conservant uniquement les arêtes dont le compteur est au moins égal à 6.

Les communautés demandées sont les composantes fortement connexes de ce graphe orienté filtré. Nous avons utilisé l’algorithme de Kosaraju-Sharir, basé sur deux parcours en profondeur : un premier parcours sur le graphe inversé pour obtenir l’ordre de traitement, puis un second parcours sur le graphe original pour construire les composantes. Notre implémentation utilise des piles explicites plutôt que de la récursion, afin d’éviter un dépassement de pile sur un grand graphe.

Pour les dix plus grandes composantes, le diamètre est calculé par BFS depuis chaque sommet de la composante, en restant dans le sous-graphe induit par cette composante. Kosaraju est en $O(V + E)$. Le diamètre exact d’une composante de taille k et contenant m arêtes internes coûte $O(k \cdot (k + m))$.

3 Résultats

3.1 Tâche 1

La tâche 1 donne 171 193 communautés pour 4 108 270 auteur·rice·s. La plus grande communauté contient 3 808 717 auteur·rice·s, tandis que 105 932 communautés sont des singletons. Les autres communautés sont beaucoup plus petites : hors composante géante, les plus grandes restent sous la taille 64.

La Figure 1 montre cette distribution avec des classes logarithmiques. On observe une forte concentration de petites communautés, puis une seule composante géante très éloignée du reste. La Figure 2, en échelle log-log, rend cette chute plus visible.

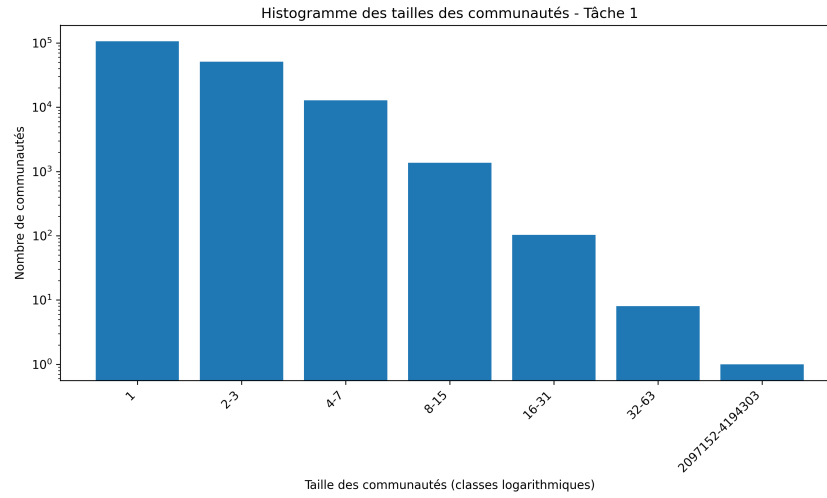


FIGURE 1 – Histogramme des tailles de communautés pour la tâche 1, avec classes logarithmiques.

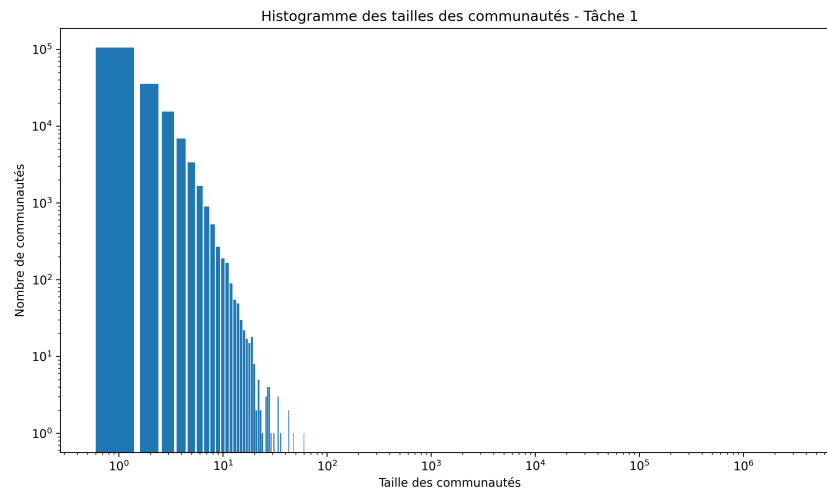


FIGURE 2 – Histogramme détaillé des tailles de communautés pour la tâche 1, en échelle log-log.

3.2 Tâche 2

La tâche 2 a été exécutée sur 8 151 751 publications. Elle donne 3 989 187 composantes fortement connexes. La très grande majorité sont des composantes de taille 1, ce qui est cohérent avec le seuil de filtrage et avec la notion de forte connexité.

La plus grande composante fortement connexe contient 565 auteur·rice·s et a un diamètre de 79. Les composantes suivantes sont nettement plus petites, comme le montre le Tableau 1.

Rang	Taille	Diamètre
1	565	79
2	38	13
3	34	7
4	33	6
5	26	8
6	21	7
7	19	12
8	18	8
9	18	8
10	18	7

TABLE 1 – Tailles et diamètres des dix plus grandes composantes fortement connexes de la tâche 2.

4 Discussion

Les résultats des deux tâches sont cohérents avec les définitions utilisées et avec les algorithmes choisis. Dans la tâche 1, l’Union-Find maintient des composantes connexes dans un graphe non orienté. Cette notion est assez large : dès qu’une publication relie plusieurs auteur·rice·s, leurs communautés fusionnent. De plus, deux auteur·rice·s peuvent appartenir à la même communauté sans avoir publié directement ensemble, tant qu’il existe une chaîne de co-publications entre eux. Il est donc logique d’observer une composante géante : dans un grand réseau scientifique, de nombreux groupes finissent par être reliés indirectement.

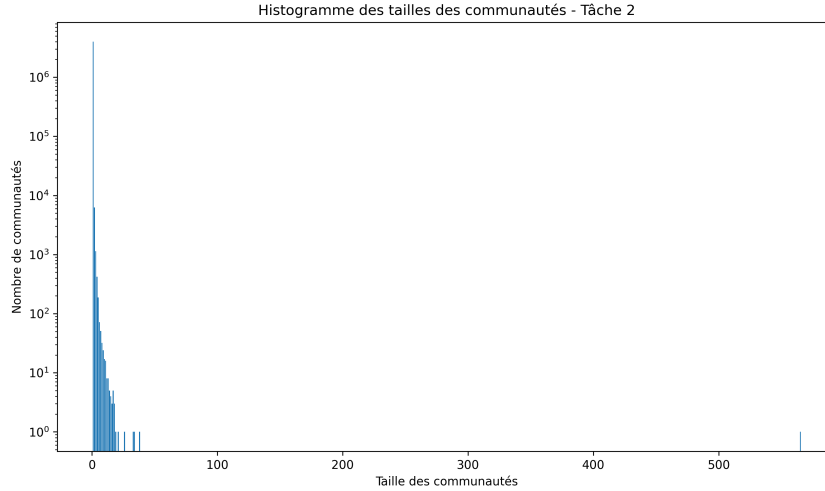


FIGURE 3 – Histogramme des tailles des composantes fortement connexes pour la tâche 2.

À l'inverse, la tâche 2 impose beaucoup plus de contraintes. D'abord, les arêtes sont orientées du premier auteur vers les autres auteur·rice·s, ce qui perd la symétrie naturelle de la co-publication. Ensuite, une arête n'est gardée que si la relation apparaît au moins six fois. Enfin, Kosaraju ne regroupe deux sommets dans une même composante que s'ils sont accessibles l'un depuis l'autre par des chemins orientés. Ces trois contraintes expliquent le grand nombre de singletons et la taille beaucoup plus limitée des communautés obtenues. Le résultat n'est donc pas contradictoire avec celui de la tâche 1 : il montre simplement que les deux tâches ne mesurent pas le même type de communauté.

Le diamètre donne aussi une information intéressante. La plus grande composante de la tâche 2 a une taille de 565, mais un diamètre de 79, ce qui suggère une structure assez étirée : certains auteur·rice·s sont relié·e·s par de longs chemins orientés, même s'ils appartiennent à la même composante fortement connexe. À l'inverse, certaines composantes plus petites ont des diamètres plus faibles, ce qui correspond davantage à des groupes compacts où les chemins entre auteur·rice·s sont courts.

Une limite de notre solution est le coût du calcul exact du diamètre. La méthode par BFS depuis chaque sommet est correcte, car le graphe n'est pas pondéré, mais elle peut devenir coûteuse pour de très grandes composantes. Pour une base plus grande comme OpenAlex, il faudrait probablement approximer les diamètres ou limiter ce calcul aux composantes réellement pertinentes. De même, le comptage des paires orientées peut consommer beaucoup de mémoire ; une optimisation serait de représenter les auteur·rice·s par des entiers dès la tâche 2, plutôt que de conserver les noms dans les structures de comptage.

5 Conclusion

La tâche 1 met en évidence une communauté géante de co-publication. Cela montre que la connexité non orientée est une notion large, qui relie facilement des groupes d'auteur·rice·s par des chaînes de collaborations. La tâche 2 donne une vision différente : avec des relations orientées, répétées au moins six fois, et une condition de forte connexité, les communautés deviennent beaucoup plus rares et plus petites. L'Union-Find est donc bien adapté à la première tâche, car il maintient efficacement des composantes connexes en ligne. Kosaraju-Sharir est adapté à la seconde, car il calcule directement les composantes fortement connexes d'un graphe orienté. Les résultats obtenus confirment que le choix du modèle de graphe influence fortement les communautés détectées.